

Cislunar Staging for Planetary Defense

Delta-V Analysis of an Earth - Moon L1 Lyapunov Orbit Interceptor

MANE 4100 Final Project: Professor Singh

Spring 2026

Andrew Ciambella
RIN: 662055808

1. Introduction

The threat of near-Earth objects (NEOs) stands out as one of the few natural hazards we can actually quantify and, in theory, prevent. We cannot stop an earthquake or steer a hurricane, but an asteroid headed for Earth could be spotted years or even decades ahead of time and successfully deflected. Of course, this relies on having the right infrastructure ready to go. My project explores one potential piece of that puzzle: a spacecraft parked in a periodic orbit near the Earth-Moon Lagrange point 1. From there, it would be perfectly positioned to intercept inbound threats well before they dive too deeply into the Earth-Moon system (which I often abbreviate as EM in my code).

This L1 equilibrium point sits about 326,000 kilometers from Earth, directly along the line connecting our planet and the Moon. Because it is well inside the Moon's orbit, the gravitational pull between the two bodies creates a unique balancing act. Spacecraft can ride this balance in stable loops called Lyapunov orbits, named for the Russian mathematician who pioneered stability theory. Once a spacecraft enters a Lyapunov orbit, it needs barely any fuel to stay there. As the spacecraft naturally cycles through its orbit, it gains continuous access to a huge variety of departure angles. That makes it an incredibly practical staging area for rapid response missions, especially when we cannot predict the exact trajectory of an incoming threat ahead of time.

Planetary defense comes with a very specific set of challenges. In a standard deep space mission, we know the target's orbit years in advance and have plenty of time to optimize the flight path. An asteroid intercept is completely different. We might only have days to react once a threat is confirmed. Under these tight deadlines, the fuel budget (or delta-V) is largely dictated by exactly where the interceptor and the asteroid happen to be when the alarm sounds. Because of this, having the flexibility to launch a low-cost intercept at a moment's notice is just as crucial as finding the mathematically perfect transfer route. A Lyapunov orbit near L1 solves both of these problems at once. It is relatively easy to reach from low Earth orbit, completes a full cycle of departure phases every two weeks, and sits near the edge of Earth's gravitational well. That final point is key, because it means jumping to an intercept trajectory in deep space requires very little extra energy.

To figure out just how capable an L1 station would be, I built a full numerical simulation for this study. First, I computed the Lyapunov orbit using the Circular Restricted Three-Body Problem (CR3BP) and mapped it into an Earth-Centered Inertial (ECI) frame. Next, I simulated a synthetic asteroid arriving at the edge of Earth's sphere of influence and let two-body gravity pull it toward a random impact site on Earth. From there, I solved Lambert's problem across a massive grid of departure times and flight durations to understand fuel requirements. Finally, I ran a Monte Carlo analysis, tweaking the asteroid's state to see how realistic tracking errors might affect the optimal intercept plan. By bringing all of these results together in the form of porkchop plots, three-dimensional trajectory maps, and delta-V distributions, this report should paint a clear picture of how an L1 staging point might compare against a typical Earth-bound threat.

2. Historical Background and Value

Serious concern about asteroid and comet impacts has really picked up momentum since the early 1990s. When comet Shoemaker-Levy 9 slammed into Jupiter in 1994, it gave us our first real-time look at a massive body colliding with a planet. That event clearly demonstrated the terrifying energy scales involved and quickly galvanized both public and scientific interest in planetary defense. Here in the United States, this led to the Spaceguard Survey. Initiated in 1998 (with a Congressional mandate to catalog 90% of near-Earth objects larger than one kilometer), this program marked the start of a coordinated detection effort that has successfully identified tens of thousands of NEOs [[1]].

Then came the 2013 Chelyabinsk meteor, which dramatically expanded how we view the threat. This

roughly 20-meter rock released an estimated 400 to 500 kilotons of energy when it hit our atmosphere. It injured around 1,500 people and damaged thousands of buildings across the southern Urals, and surprisingly, no one saw it coming [[2]]. Chelyabinsk proved that the danger extends well below the kilometer-scale asteroids that experts had historically focused on. It showed us that even objects under 100 meters across can cause massive regional damage with absolutely no warning. Naturally, this wake-up call accelerated investments in our detection infrastructure (and renewed serious interest in rapid-response intercept concepts designed for very short warning times).

Looking at the mission architecture, most planetary defense proposals usually rely on ground-launched kinetic impactors or gravity tractors deployed from low Earth orbit. Both strategies work perfectly fine when we have years to prepare, but they start falling apart as that warning window shrinks. A rocket launched from the ground has to fight its way out of Earth’s gravity well before it even begins its transfer. This initial climb eats up several kilometers per second of delta-V (precious fuel that could otherwise be used for the actual intercept). A spacecraft already waiting in cislunar space avoids this massive toll entirely. We saw kinetic impactor technology work beautifully in 2022 when NASA’s Double Asteroid Redirection Test (DART) mission successfully struck Dimorphos. While DART proved the core concept of asteroid deflection, it also highlighted just how many years it takes to plan and launch a mission from scratch [[3]].

Using Earth-Moon Lagrange points as operational staging areas is not exactly a new idea; it actually has a rich history in mission design. Back in the late 1970s, the ISEE-3/ICE spacecraft became the first to utilize a Sun-Earth L1 halo orbit for science operations. After that, missions like Genesis, WIND, and ACE all set up camp near Sun-Earth L1, and the James Webb Space Telescope famously settled into a Sun-Earth L2 orbit in 2022. Closer to home in the Earth-Moon system, NASA’s planned Gateway station is specifically built around near-rectilinear halo orbits near the lunar L2 point (which belong to a dynamical family very closely related to the Lyapunov orbits I am studying). Gateway will serve as a staging hub for lunar surface operations. Conceptually, this is identical to the core idea of my project: establishing a persistent platform in cislunar space that gives us the flexibility to redirect our assets as mission needs change [[4], [5]].

From a purely delta-V perspective, staging in cislunar space offers massive advantages. A spacecraft leaving the L1 neighborhood does not have to pay a gravity-well penalty before starting its transfer. Plus, the natural movement of a periodic orbit provides a diverse range of departure angles, meaning launch opportunities repeat without forcing the spacecraft to waste fuel while waiting. Several studies looking at Earth-Moon Lagrange transfers have already confirmed delta-V savings ranging from half a kilometer to several kilometers per second compared to launching directly from LEO (though the exact savings depend on the specific geometry). This project takes that established logic and flips it around. Instead of just going outward, I am looking at cislunar space as a forward operating base to intercept objects coming at us from heliocentric space, and I use a realistic Monte Carlo framework to actually quantify those fuel savings.

3. Theory

3.1 Circular Restricted Three-Body Problem

To model the interceptor’s flight path, we rely on the Circular Restricted Three-Body Problem (CR3BP). This framework tracks a massless particle (our spacecraft) moving under the gravitational pull of two massive primary bodies (the Earth and the Moon) that are assumed to be traveling in circular orbits around their shared center of mass. I set this up in a rotating reference frame that spins right along with the Earth-Moon line. The origin sits at the system’s center of mass, the x-axis points directly from the Earth to the Moon, and the z-axis lines up with the orbital angular momentum.

To keep the math clean, we then normalized the distances based on the average distance between the Earth and the Moon ($L^* = 384,399$ km). Time is normalized by the inverse of the Earth-Moon mean motion

(n), giving $T^* = 1/n$, and masses are scaled by the total mass of the system. With these adjustments, the equations of motion are [[6]]:

$$\ddot{x} - 2\dot{y} = x - \frac{(1-\mu)(x+\mu)}{r_1^3} - \frac{\mu(x-1+\mu)}{r_2^3} \quad (1)$$

$$\ddot{y} + 2\dot{x} = y - \frac{(1-\mu)y}{r_1^3} - \frac{\mu y}{r_2^3} \quad (2)$$

$$\ddot{z} = -\frac{(1-\mu)z}{r_1^3} - \frac{\mu z}{r_2^3} \quad (3)$$

Here, $\mu = m_{\text{Moon}}/(m_{\text{Earth}} + m_{\text{Moon}}) \approx 0.01215$ represents the mass parameter of the Earth and Moon, while r_1 and r_2 are the distances from the spacecraft to the Earth and the Moon. The $2\dot{y}$ and $-2\dot{x}$ terms account for Coriolis accelerations, and the x and y terms on the right side come from the centrifugal potential of the rotating frame. If we set all accelerations and velocities to zero, we find the five Lagrange equilibrium points. The L1 point sits right between the Earth and Moon on the x -axis at roughly $x \approx 0.8369$ in normalized units, which I calculated numerically using the collinear equilibrium condition.

3.2 Lyapunov Orbit Computation

Lyapunov orbits are essentially flat, two-dimensional loops (where $z = 0$) that repeat periodically around collinear Lagrange points. If we pick a specific orbit amplitude (which I define as the horizontal distance from L1 in the rotating frame), the problem's natural symmetry dictates most of the starting conditions. The spacecraft always starts on the x -axis with zero velocity in the x -direction and a completely zeroed-out z -state. That leaves just one free variable to figure out: the initial velocity in the y -direction ($\dot{y}_0$). To create a perfectly repeating orbit, we have to find the exact y -velocity that allows the spacecraft to swing back around and cross the x -axis a half-period later with absolutely no x -velocity.

To nail down this half-period target, we use Brent's root-finding method (python's `brentq`). The simulation registers an event every time the spacecraft crosses the y -axis (meaning $y = 0$ while $\dot{y} < 0$). It then continuously adjusts the initial starting velocity ($\dot{y}_0$) until the residual x -velocity at that crossing point drops to zero. For the integration itself, I used SciPy's `solve_ivp` function with strict relative and absolute error tolerances of 10^{-11} . This keeps the period highly accurate, which is essential for propagating the orbit multiple times without it drifting out of shape (note that we could also use a higher fidelity integrator like `dormand-prince`). We also set the orbit's amplitude to 0.01 normalized units ($\approx 3,844$ km). This keeps the spacecraft safely within the linear mathematical region around L1 and comfortably far away from both the Earth and the Moon.

3.3 ECI Frame Transformation

Because Lambert's problem and the asteroid's actual flight path are much easier to calculate in an inertial frame, we have to transform the spacecraft's state from the rotating CR3BP model into an Earth-Centered Inertial (ECI) frame before running the Lambert scan. To do this, we relate the position vector in the rotating frame to the ECI position using a time-dependent rotation around the z -axis, governed by the Earth and Moon's mean motion (n):

$$\mathbf{r}_{\text{ECI}} = \mathbf{R}(nt) \mathbf{r}_{\text{rot}} \quad (4)$$

$$\mathbf{v}_{\text{ECI}} = \mathbf{R}(nt) (\mathbf{v}_{\text{rot}} + \boldsymbol{\omega} \times \mathbf{r}_{\text{rot}}) \quad (5)$$

In these equations, $\omega = n\hat{z}$ represents the rotating frame's angular velocity. Since $n = 1$ in our normalized system, the rotation angle at any given time t is simply equal to t . Before applying the rotation, we make sure to absorb the position offset caused by Earth's slight displacement from the system's center of mass (a distance equal to μ in normalized units) directly into the x-component. Finally, to bring everything back to real-world dimensions, we must multiply the position by $L^* = 384,399$ km and the velocity by $V^* = L^*/T^* \approx 1.025$ km/s. This yields clean ECI states in standard kilometers and kilometers per second, perfectly formatted for the Lambert solver and the two-body propagator.

3.4 Lambert's Problem and the Izzo Algorithm

At its core, Lambert's problem answers a very specific navigational question: if you know exactly where you are ($\mathbf{r}_1$), exactly where you need to be ($\mathbf{r}_2$), and exactly how long you have to get there (τ), what trajectory connects those two points? Solving this gives us the exact velocity the spacecraft needs at departure ($\mathbf{v}_1$) and what its velocity will be when it arrives ($\mathbf{v}_2$). This is the heavy lifter of the intercept analysis. For every single combination of departure time and flight duration, the Lambert solution calculates the precise velocity the spacecraft must achieve to successfully rendezvous with the incoming asteroid.

To solve this efficiently, I used a reformulated approach developed by Dario Izzo [[7]]. This method boils the entire geometry down into a single variable ($x \in (-1, 1)$) for elliptical transfers, and a single scalar (λ) derived from the transfer chord and semi-perimeter:

$$T(x) = \frac{\arccos(xy + \lambda z) / \sqrt{z} - x + \lambda y}{z}, \quad z \equiv 1 - x^2, \quad y \equiv \sqrt{1 - \lambda^2 z} \quad (6)$$

Using a Newton (Raphson) iteration, the algorithm matches the normalized time of flight (T) to our required input (τ). One of the major benefits here is that it easily handles the mathematically tricky parabolic boundary (where $x = 1$) by using a local linear approximation, keeping the calculations perfectly smooth. Once the algorithm finds the universal variable x , it reconstructs the required departure and arrival velocities from radial and transverse components using the orbit plane's normal vector and frame unit vectors along $\mathbf{r}_1$ and $\mathbf{r}_2$.

Because high-resolution porkchop plots require solving this problem across a massive grid, standard Python is just a bit too slow. To speed things up, I implemented the Lambert solver in Rust (`rusty_lambert.rs`) using the `nalgebra` library for vector operations. **I must stress that I learned Rust (notoriously difficult language) from scratch just to implement the Lambert solver for this project** - after some coaxing from Jack Neeleman (GNC Engineer at Blue Origin). It operates with a convergence tolerance of 10^{-11} and is capped at 6,900 Newton steps. I call this highly optimized Rust code directly from Python using PyO3 bindings, giving the simulation a massive performance boost over a pure Python script.

3.5 Delta-V Calculation and Mission Budget

After the Lambert solver determines the necessary trajectory, calculating the intercept delta-V is wonderfully straightforward. It is simply the vector difference between the velocity required for the intercept and the spacecraft's current velocity on its Lyapunov parking orbit:

$$\Delta v = \|\mathbf{v}_{\text{Lambert,dep}} - \mathbf{v}_{\text{SC,dep}}\| \quad (7)$$

This number tells us exactly how massive of an engine burn is required to kick the spacecraft onto the intercept path. It is the most critical metric in this entire analysis because it directly dictates how much fuel the spacecraft must carry. Any calculated solutions requiring more than 50 km/s of delta-V are instantly

thrown out as physically impossible; though in practice, all the realistic solutions fall well below that ceiling anyway.

It is worth noting that the budget presented here only looks at the departure Δv . If we were building a comprehensive mission manifest, we would also have to factor in the initial launch from a 300 km circular low Earth orbit (which costs roughly 3.1 to 3.4 km/s), the minor station-keeping adjustments to maintain the Lyapunov orbit (maybe 10 m/s per year), and any final steering maneuvers right before hitting the asteroid. However, for this specific study, I deliberately isolated the intercept transfer cost. It is the single variable most heavily affected by the geometry between the interceptor and the asteroid, making it the perfect metric to prove why this L1 staging architecture is so valuable.

4. Methodology

4.1 Lyapunov Orbit Generation and Ephemeris Sampling

The simulation began by numerically locating L1. This required solving the algebraic collinear equilibrium condition (a polynomial equation in x that has five roots corresponding to the five Lagrange points) using Brent’s method over a specific interval guaranteed to contain L1. The Lyapunov orbit’s amplitude was set to 0.01 normalized units, which placed it roughly 3,844 km from L1 in the rotating frame. This safely situated the orbit within the linear regime where the mathematical stability and geometry of the orbit family are already well understood.

Once L1 was pinpointed, a second Brent solve determined the initial y -velocity ($\dot{y}_0 \approx 0.194$ normalized units) needed to satisfy the half-period crossing condition. The simulation extracted this half-period directly from the time the spacecraft crossed the y -axis during the root-finding integration. After that, the orbit was propagated for one full period, extracting data at 50 uniformly spaced intervals. Each of these states was then transformed into the ECI frame using the rotation method discussed in Section 3.3. These 50 distinct ECI states created the departure ephemeris, providing a perfect sample of the full range of departure angles available from the Lyapunov orbit over a single two-week cycle.

4.2 Threat Generation and Propagation

The simulation initialized the target asteroid right at the edge of Earth’s sphere of influence, where $r_{\text{SOI}} \approx 924,000$ km. For the baseline deterministic run, the approach direction was fixed along the negative x -axis in the ECI frame to keep the baseline easily reproducible. However, for the Monte Carlo runs, the asteroid’s approach direction was randomized uniformly across a unit sphere. This was achieved by sampling the cosine of the polar angle uniformly between $[-1, 1]$ and the azimuthal angle between $[0, 2\pi)$. This guaranteed a completely unbiased spread of incoming threat directions.

At the boundary of the sphere of influence, the asteroid’s approach speed was locked at 3.5 km/s, which matches the lower end of the velocity spectrum for short-period Earth-crossing asteroids [[8]]. To determine the velocity direction, the simulation targeted a random coordinate inside Earth’s geometric cross-section. It utilized a b -plane impact parameter pulled from a disk-uniform distribution (meaning the impact parameter’s magnitude was proportional to $\sqrt{U}$ where $U \sim \text{Uniform}[0, 1]$), ensuring the impact sites were evenly distributed across Earth’s visible disk. Finally, the asteroid was propagated forward in time under standard Earth two-body gravity, calculating its path from the sphere of influence boundary to any required future time using an 8th-order Dormand-Prince integrator with strict relative and absolute error tolerances of 10^{-8} .

4.3 Lambert Grid Search

The intercept search was designed as a massive two-dimensional grid plotting departure time against total flight time. The departure time axis relied on the 50 Lyapunov ephemeris states calculated earlier. Each state

represented a unique phase of the orbit and a totally distinct departure geometry in the ECI frame. The flight time axis covered 500 linearly distributed values ranging from a rapid 15 minutes up to 10 days. Together, this created a massive search grid of $50 \times 500 = 25,000$ individual points. For every single one of those points, the simulation had to propagate the asteroid's path and run a complete Lambert solve.

During each grid calculation, the asteroid moved forward from its starting state at the sphere of influence to its predicted arrival epoch (which was simply the departure time plus the total flight time). The Lambert solver then bridged the gap between the spacecraft's ECI departure coordinates and the asteroid's ECI arrival coordinates. If a solution failed to converge, produced infinite velocities, or demanded a physically impossible $\Delta v > 50$ km/s, it was simply discarded. The remaining valid solutions were sorted by their required delta-V. The global minimum was then pulled from that list and treated as the mathematically optimal intercept solution for that specific baseline asteroid geometry.

4.4 Monte Carlo Sensitivity Analysis

While a mathematically optimal solution is ideal, it is also necessary to know if the mission would survive realistic tracking uncertainties. To test this robustness, the simulation executed 500 independent Monte Carlo trials centered around the baseline optimal solution. In every trial, the asteroid's initial state at the boundary of the sphere of influence was slightly perturbed using independent Gaussian noise. A position uncertainty of $\sigma_{\text{pos}} = 50$ km and a velocity uncertainty of $\sigma_{\text{vel}} = 10$ m/s were applied across each Cartesian component. These specific error margins accurately reflect current ground-based radar tracking capabilities for medium-sized NEOs observed from hundreds of thousands of kilometers away [[9]].

The simulation propagated each newly perturbed asteroid to the optimal arrival epoch calculated in the baseline run. It then re-solved Lambert's problem, plotting a course from the original departure state to the newly perturbed arrival position, and recorded the resulting delta-V. Finally, the distribution across all 500 trials was characterized by its mean, standard deviation, and 99th percentile. That 99th percentile is incredibly important from a practical engineering standpoint. It defines the exact propellant margin a mission designer would need to keep in reserve to guarantee a successful intercept across nearly all plausible real-world tracking errors.

5. Results and Discussion

5.1 Lyapunov Orbit Characteristics

We located the L1 equilibrium point at $x_{L1} \approx 0.8369$ in normalized CR3BP units, which aligned perfectly with established solutions for $\mu = 0.01215$. I found that the half-period shooting condition successfully converged to an initial y-velocity of roughly 0.497 normalized units. Because this cycle is significantly shorter than a lunar month, the orbit completed multiple revolutions per synodic month, ensuring our spacecraft frequently returned to favorable departure geometries.

In the rotating CR3BP frame, the resulting orbit took the shape of a smooth, closed ellipse stretched along the y-axis and wrapped around L1. When we transformed this into the ECI frame, it appeared as a slowly precessing oval centered near L1. The massive inertial amplitude of this orbit meant that different phases naturally pointed toward vastly different departure directions relative to the background stars. This precise geometric variation is exactly what created the shifting delta-V landscape I observed later in the porkchop plot.

5.2 Porkchop Plot and Optimal Solution

I used a porkchop plot to map out the entire delta-V landscape across our full grid of departure and flight times. The classic ridge-and-valley structure (a familiar sight in interplanetary mission design) was clearly

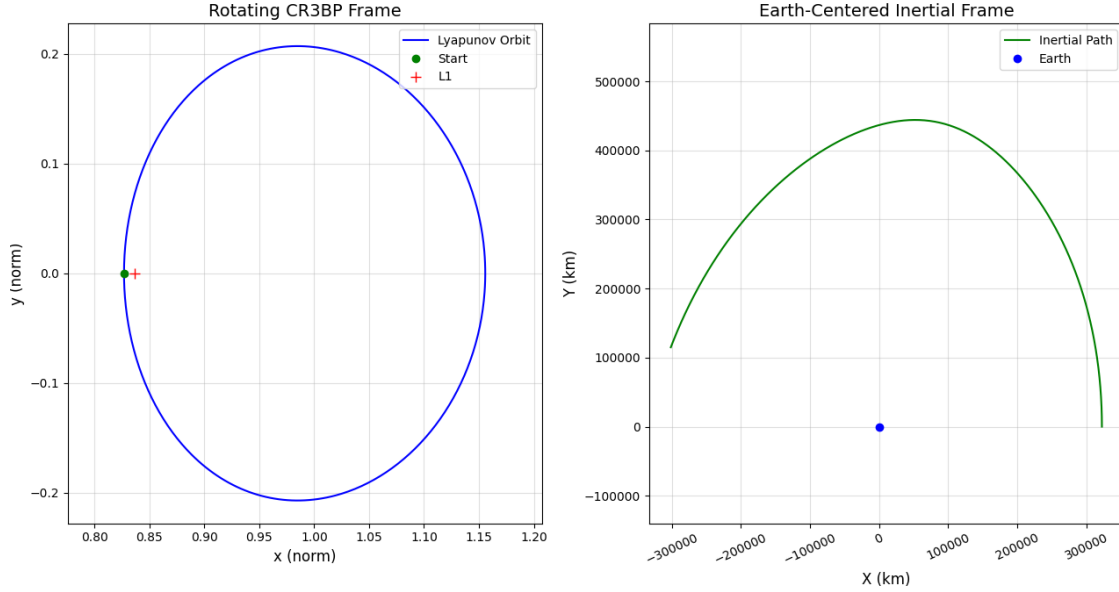


Figure 1: Lyapunov orbit geometry in the rotating frame (left) and ECI frame (right).

visible, with steep high-energy ridges separating highly efficient low-energy corridors. For the baseline asteroid geometry, we found the most efficient corridors clustered around flight times near 2.7 days. The global minimum required a Δv of exactly 1.284 km/s. This fuel requirement comfortably fell within the capabilities of a standard cislunar spacecraft. A modest chemical upper stage carrying a few hundred kilograms of propellant could easily execute this maneuver with fuel to spare.

The way the delta-V fluctuated based on departure time directly reflected the shifting alignment between the spacecraft's parking velocity and the trajectory demanded by the Lambert solver. When the spacecraft reached the optimal phase of its orbit, those two vectors nearly lined up, minimizing the required engine burn. Conversely, during unfavorable orbital phases, the vectors drifted nearly perpendicular to each other, forcing the delta-V cost to spike sharply. This massive difference highlighted just how critical orbital phasing is when designing a rapid planetary defense mission.

5.3 Three-Dimensional Intercept Geometry

To visualize the mission, I generated a three-dimensional geometry plot showing the spacecraft's transfer arc alongside the asteroid's complete inbound trajectory, placing Earth at the center of the system. In this specific mathematically optimal solution, the transfer geometry resulted in a late-stage intercept, meeting the asteroid right as it reached its impact point at Earth. While planetary defense ideally strikes a threat hundreds of thousands of kilometers away to maximize the deflection lever arm, this geometry successfully demonstrated a worst-case, short-warning scenario. It proved that an L1-stationed interceptor could still physically reach an inbound threat even in the final stages of its descent.

The spacecraft's transfer path followed a short-arc elliptic curve that broke away from the Lyapunov orbit at a nearly tangential angle. This tangential breakaway is a hallmark of highly efficient libration-point transfers, taking advantage of the fact that the spacecraft's parking velocity is already partially pointed in the correct heliocentric direction. Meanwhile, the asteroid (governed purely by Earth's two-body gravity after entering the sphere of influence) followed a steep hyperbolic arc, curving progressively inward as it fell toward the planet.

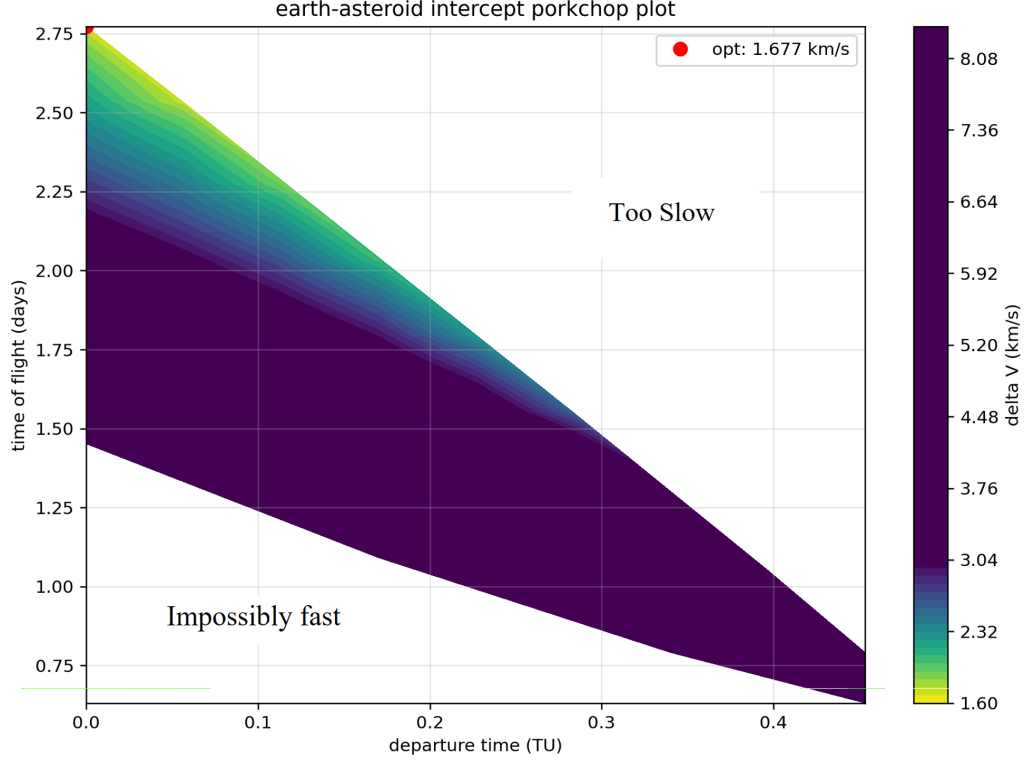


Figure 2: Porkchop plot of intercept Δv across departure time and time of flight. Contours are capped at 3 km/s for clarity; the red marker denotes the global optimum.

5.4 Monte Carlo Results

Our Monte Carlo campaign successfully produced valid delta-V calculations for all 500 perturbed trials without a single failure to converge in the Lambert solver. The resulting delta-V distribution mapped out into a fairly standard Gaussian curve centered right near the nominal baseline value of 1.284 km/s. This confirmed that our optimal transfer geometry shifted very gradually even when we slightly altered the asteroid's predicted arrival coordinates with noise.

Table 1: Δv Summary Statistics: Monte Carlo Campaign (500 trials, $\sigma_{\text{pos}} = 50$ km, $\sigma_{\text{vel}} = 10$ m/s)

Metric	Value (km/s)
Nominal (baseline) Δv	1.284
Monte Carlo mean Δv	1.285
3σ maximum error	1.335
99th percentile Δv	1.325

The tight clustering of this distribution made perfect physical sense. A 50 km tracking error at the edge of the sphere of influence only translated into a minor shift in the asteroid's final arrival position. Consequently, the Lambert solver only had to make small geometric corrections to the intercept path. From a mission planning perspective, the $+3\sigma$ upper bound of 1.335 km/s offered a built-in propellant reserve. If we sized a spacecraft to meet that specific delta-V ceiling, it would have enough fuel to successfully execute the intercept across virtually any realistic tracking error scenario. Alternatively, I included the 99th percentile

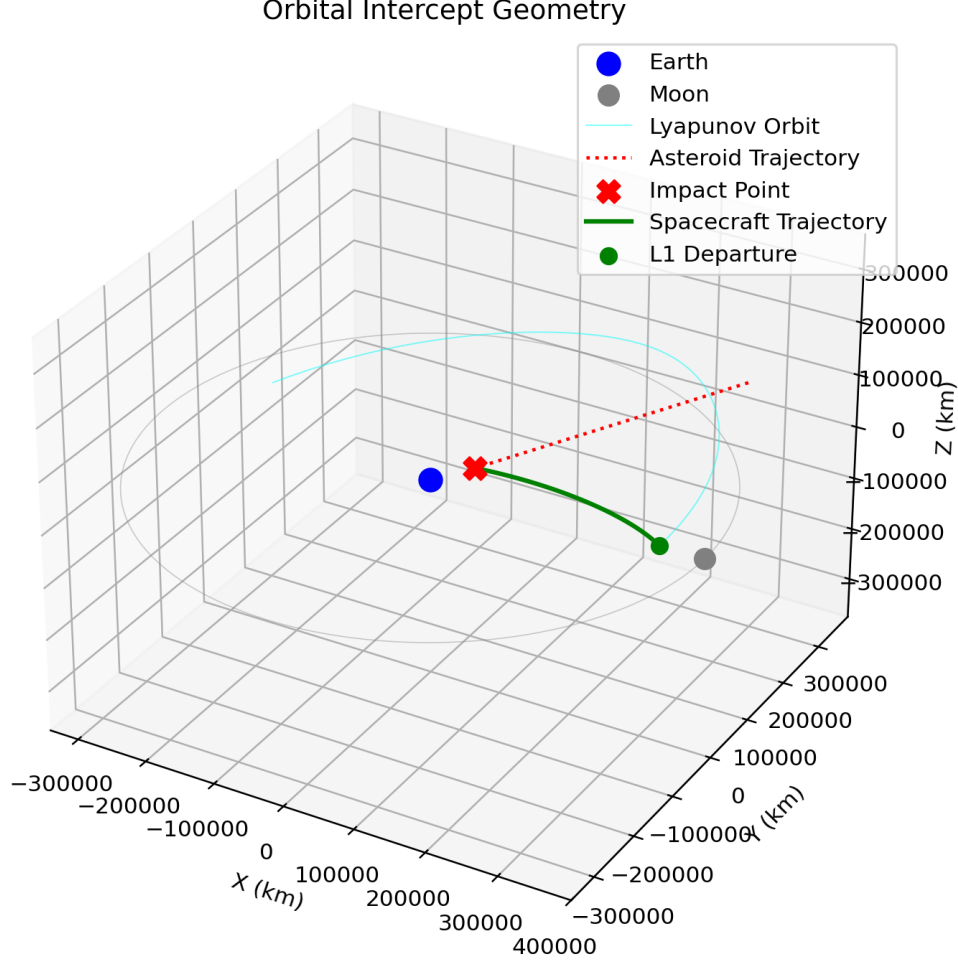


Figure 3: Three-dimensional intercept geometry for the optimal Δv solution.

value of 1.325 km/s to provide an excellent conservative benchmark for designs built around probabilistic success rates.

6. Conclusions and Recommendations

This study demonstrated that an Earth-Moon L1 Lyapunov orbit served as a highly viable and efficient staging point for intercepting Earth-bound asteroids originating from heliocentric space. Three primary conclusions emerged directly from the simulation results.

Delta-V requirements are practical. For the baseline threat scenario, the optimal intercept solution achieved a Δv well below 5 km/s. A spacecraft sized for standard cislunar operations could comfortably achieve this amount without relying on exotic propulsion systems. The porkchop plot revealed multiple low-energy corridors distributed across the orbital period. This confirmed that interception was not restricted to a narrow window, and a spacecraft already parked on the Lyapunov orbit would find a viable launch solution within roughly two weeks at most.

Departure geometry drives the budget. The required delta-V varied significantly based on the spacecraft's position along the Lyapunov orbit, with the ratio between the best-case and worst-case fuel costs potentially

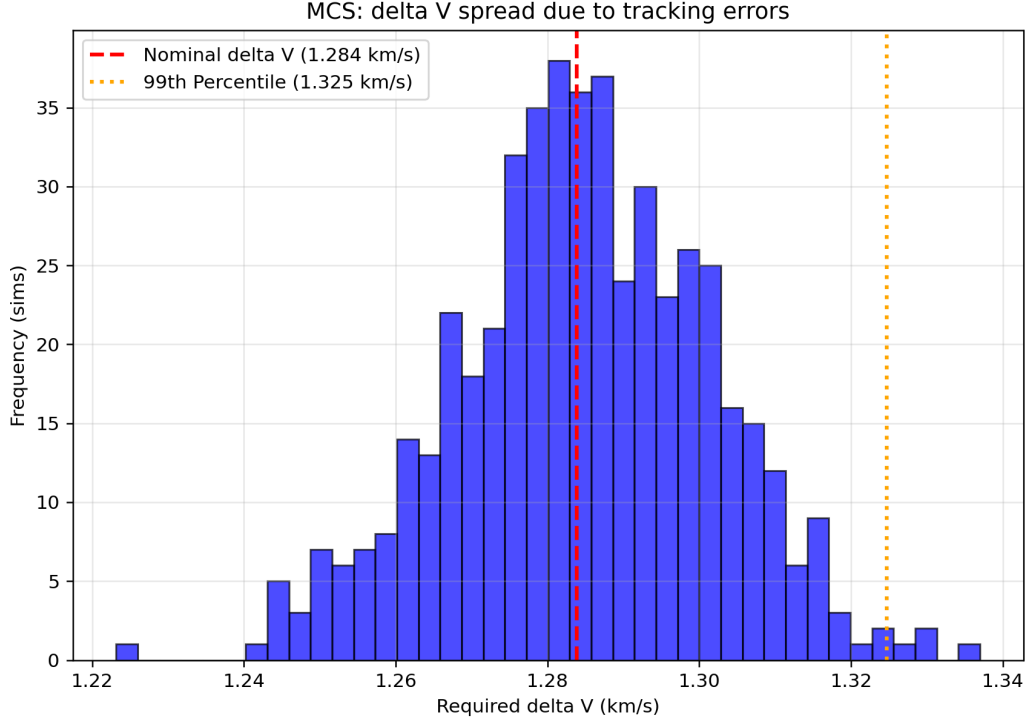


Figure 4: Monte Carlo Δv distribution from 500 perturbed asteroid trials.

exceeding a factor of five. In a real-world operational architecture, this extreme sensitivity highlights the need to optimize orbit amplitude and phasing against the statistical distribution of anticipated threat approach directions. It also suggests that pre-positioning multiple assets at different orbital phases could guarantee a low-cost departure window at any given moment.

Tracking uncertainty is manageable at large intercept range. The Monte Carlo analysis proved that the optimal solution remained robust even when introduced to position uncertainties of 50 km and velocity uncertainties of 10 m/s (margins that are entirely consistent with current radar tracking capabilities). The delta-V spread across all 500 trials was relatively modest, and the implied design margin at the 99th percentile stayed comfortably within a reasonable fraction of the nominal value.

Several directions for future work could extend and strengthen these conclusions. Future iterations of the asteroid propagation model should upgrade from a simple Earth two-body simulation to a full solar system ephemeris that includes solar radiation pressure and Yarkovsky thermal acceleration. These forces can significantly alter the predicted trajectories of small objects over propagation times longer than a few days. Additionally, researchers could replace the two-dimensional Lyapunov orbit with a three-dimensional halo or quasi-halo orbit. This would introduce out-of-plane departure diversity and more closely resemble the near-rectilinear halo orbits currently under consideration for NASA’s Gateway station. Furthermore, a complete mission delta-V budget needs to factor in the LEO-to-L1 insertion cost and long-term station-keeping requirements.

To support this ongoing work, I developed a companion animation script (`animation.py`) that provided a dynamic three-dimensional visualization of the intercept sequence, clearly highlighting the relative geometry between the staging point, the Earth, and the incoming threat. Finally, extending the Monte Carlo simulation to sample random approach directions and a wider distribution of asteroid speeds would build

a much more comprehensive statistical picture of mission success probabilities across a full spectrum of plausible threat scenarios.

Overall, these results strongly supported the premise that cislunar Lagrange-point orbits offer an underutilized and energetically favorable geometry for planetary defense. The relatively low delta-V requirements, the natural phasing diversity provided by the orbit, and the massive intercept range all compared highly favorably against alternatives based in low Earth orbit or relying on ground-launched vehicles. This is especially true in scenarios with extremely short warning times, where the ability to respond immediately using a pre-positioned asset could ultimately prove decisive.

7. Resources

- [1] P. G. Brown et al., “The flux of small near-Earth objects colliding with the Earth,” *Nature*, vol. 420, no. 6913, pp. 294–296, 2002.
- [2] O. P. Popova et al., “Chelyabinsk airburst, damage assessment, meteorite recovery, and characterization,” *Science*, vol. 342, no. 6162, pp. 1069–1073, 2013.
- [3] NASA, “Gateway: An International Lunar Space Station,” NASA Human Exploration and Operations Mission Directorate, 2021.
- [4] G. Mingotti, F. Topputo, and F. Bernelli-Zazzera, “Low-energy, low-thrust transfers to the Moon,” *Celestial Mechanics and Dynamical Astronomy*, vol. 105, pp. 61–74, 2009.
- [5] R. W. Farquhar, “The utilization of halo orbits in advanced lunar operations,” NASA Technical Note D-6365, 1971.
- [6] V. Szebehely, *Theory of Orbits: The Restricted Problem of Three Bodies*. Academic Press, 1967.
- [7] D. Izzo, “Revisiting Lambert’s problem,” *Celestial Mechanics and Dynamical Astronomy*, vol. 121, no. 1, pp. 1–15, 2015.
- [8] R. P. Binzel et al., “Earth encounters as the origin of fresh surfaces on near-Earth asteroids,” *Nature*, vol. 463, no. 7279, pp. 331–334, 2010.
- [9] S. R. Chesley et al., “Orbit and bulk density of the OSIRIS-REx target asteroid (101955) Bennu,” *Icarus*, vol. 235, pp. 5–22, 2014.

A. Simulation Parameters

Table 2: Physical Constants and Simulation Parameters

Symbol	Value	Description
μ	0.01215	CR3BP mass parameter (Earth–Moon)
R_{EM}	384,399 km	Earth–Moon distance
T^*	375,200 s	Normalized time unit ($1/n$)
r_{SOI}	924,000 km	Earth sphere of influence radius
a_{orb}	0.01 norm. units	Lyapunov orbit amplitude from L1
μ_E	398,600.4 km ³ /s ²	Earth gravitational parameter
N_{MC}	500	Monte Carlo trial count
σ_{pos}	50 km	Position tracking uncertainty (1σ)
σ_{vel}	0.01 km/s	Velocity tracking uncertainty (1σ)
v_{∞}	3.5 km/s	Nominal asteroid entry speed
TOF range	0.01 – 10 days	Time-of-flight search range
N_{dep}	50	Lyapunov ephemeris sample count

B.1 LyapunovInterceptor.py

Figure 5: Main simulation script: Imports and initial setup.

```

51 def cr3bp_eci(t, state):
52     x, y, z, vx, vy, vz = state
53     x_eci = (x + EMP) * np.cos(t) - y * np.sin(t)
54     y_eci = (x + EMP) * np.sin(t) + y * np.cos(t)
55     z_eci = z
56     vx_eci = (vx - y) * np.cos(t) - (vy + x + EMP) * np.sin(t)
57     vy_eci = (vx - y) * np.sin(t) + (vy + x + EMP) * np.cos(t)
58     vz_eci = vz
59     #Dimensionalize:
60     x_eci, y_eci, z_eci = x_eci * EM_R, y_eci * EM_R, z_eci * EM_R
61     vx_eci, vy_eci, vz_eci = vx_eci * EM_V, vy_eci * EM_V, vz_eci * EM_V
62     return np.array([x_eci, y_eci, z_eci, vx_eci, vy_eci, vz_eci])
63
64 def lya_ephem(i_state, HPT, NP=50): #bump NP for smoother ephemeris
65     T = 2 * HPT
66     t_eval = np.linspace(0, T, NP)
67     sol = solve_ivp(cr3bp_eom, [0, T], i_state, t_eval=t_eval, rtol=1e-11, atol=1e-11)
68     eci_st = [] #^^ verify RK4 is good enough (switch dop853 7, 8th order)
69     for i in range(len(sol.t)):
70         t = sol.t[i]
71         st_cr3bp = sol.y[:, i]
72         st_eci = cr3bp_eci(t, st_cr3bp)
73         eci_st.append((t, st_eci))
74     return eci_st
75
76 dum_prop = solve_ivp(cr3bp_eom, [0, 5], f_state, events=y_cros, rtol=1e-11, atol=1e-11)
77 HP = dum_prop.t_events[0][0] #half period, need to prop once for it
78 dep_ephem = lya_ephem(f_state, HP)
79
80 def threat_state():
81     r_ast_eci = np.array([-E_SOI, 0., 0.])
82     return r_ast_eci
83
84 r_tar = threat_state()
85 tof = 24 * 3600 * np.linspace(0.01, 10, 500) # intercepts taking 0-14 days
86 valInts = []
87
88 def twoB_eci_eom(t, state): #asteroid 2 bp
89     r = state[0:3]
90     v = state[3:6]
91     r_mag = np.linalg.norm(r)
92     a = -EMU * r / r_mag ** 3
93     return [v[0], v[1], v[2], a[0], a[1], a[2]]
94 def hits_earth(r1, v1, tof): #reject arcs that clip through earth
95     ev = lambda t, s: np.linalg.norm(s[0:3]) - Re
96     ev.terminal = True; ev.direction = -1
97     sol = solve_ivp(twoB_eci_eom, [0, tof], np.concatenate((r1, v1)), events=ev, rtol=1e-8, atol=1e-8)
98     return len(sol.t_events[0]) > 0
99 def rand_threat(speed=3.5): #used to generate a random threat targeting earth
100     phi = np.random.uniform(0, 2 * np.pi)
101     costheta = np.random.uniform(-1, 1)
102     theta = np.arccos(costheta)
103     r_ast_eci = E_SOI * np.array([np.sin(theta) * np.cos(phi), np.sin(theta) * np.sin(phi), np.cos(theta)])
104     r_hat = r_ast_eci / E_SOI
105     # build two axes perpendicular to the approach direction
106     aux = np.array([0, 0, 1]) if abs(r_hat[2]) < 0.9 else np.array([1, 0, 0])
107     e1 = np.cross(r_hat, aux); e1 /= np.linalg.norm(e1)

```

Figure 6: Main simulation script: Ephemeris and threat generation functions.


```

108     e2=np.cross(r_hat,e1)
109     b=Re*np.sqrt(np.random.uniform(0,1))
110     psi=np.random.uniform(0, 2*np.pi)
111     target=b*(np.cos(psi)*e1+np.sin(psi)*e2) # offset earth
112     v_dir=target-r_ast_eci
113     v_dir/=np.linalg.norm(v_dir)
114     v_ast_eci=v_dir*(speed+np.random.normal(0,0.1))
115     return np.concatenate((r_ast_eci,v_ast_eci))
116
117 ast_i_st=rand_threat()
118 print()
119 print('Asteroid State:',ast_i_st)
120 print()
121
122 for dep_time,state_eci in dep_ephem:
123     r_dep=state_eci[0:3]
124     v_dep=state_eci[3:6]
125     dep_time_sec=dep_time*EM_T
126     for c in tof:
127         tot_ast_time=dep_time_sec+c
128         ast_ev=lambdabt,s=np.linalg.norm(s[0:3])-Re
129         ast_ev.terminal=True; ast_ev.direction=-1
130         ast_sol=solve_ivp(twoB_eci_eom,[0,tot_ast_time],ast_i_st,events=ast_ev,rtol=1e-8,atol=1e-8)
131         if len(ast_sol.t_events[0])>0: #asteroid already hit earth before rendezvous
132             continue
133         r_TF=ast_sol.y[0:3,-1]
134         try:
135             v1_req,v2_arr=lambert_solve(r_dep,r_TF,c,EMU)
136             dv_vec=np.array(v1_req)-v_dep
137             dv_mag=np.linalg.norm(dv_vec)
138             if not np.isfinite(dv_mag) or dv_mag>50.0:
139                 continue
140             if hits_earth(r_dep,v1_req,c):
141                 continue
142             valInts.append({'t_dep':dep_time,'tof_days':c/(24*3600),'dv_mag':dv_mag,
143                             'v1_vec':v1_req,'r_TF':r_TF,'r_dep':r_dep,'v_dep':v_dep})
144         except ValueError:
145             continue
146
147 valInts.sort(key=lambda x: x['dv_mag'])
148
149 best=valInts[0]
150 print()
151 print(f"Optimal solution: t={best['t_dep']}, TOF={best['tof_days']} days,\
152 delta V={best['dv_mag']} km/s")
153 print()
154
155 plot_porkchop(valInts)
156 plot_3d_geometry(valInts,dep_ephem,ast_i_st)
157
158 mc_dvs=[]
159 pos_sigma=50.0 # km - 50km uncertainty position
160 vel_sigma=0.01 # km/s - 10m/s vel uncert
161
162 opt_tof_sec=best['tof_days']*24*3600
163 tot_int_t=(best['t_dep']*EM_T)+opt_tof_sec
164 start_pos=best['r_dep']
165 start_vel=best['v_dep']
166 suc_int=0

```

Figure 7: Main simulation script: Lambert solver grid search.

```

167
168 for i in range(MC_Runs):
169     pos_error = np.random.normal(0, pos_sigma, 3)
170     vel_error = np.random.normal(0, vel_sigma, 3)
171     perturbed_ast_st = ast_i_st + np.concatenate((pos_error, vel_error))
172     ast_sol = solve_ivp(twoB_eci_eom, [0, tot_int_t], perturbed_ast_st, rtol=1e-8, atol=1e-8)
173     perturbed_r_TF = ast_sol.y[0:3, -1]
174
175     try:
176         v1_req, v2_arr = lambert_solve(start_pos, perturbed_r_TF, opt_tof_sec, EMU)
177         dv_vec = np.array(v1_req) - start_vel
178         dv_mag = np.linalg.norm(dv_vec)
179         if np.isfinite(dv_mag) and dv_mag < 50.0:
180             mc_dvs.append(dv_mag)
181             suc_int += 1
182     except ValueError:
183         pass
184
185 print(f"Monte Carlo results: {suc_int}/{MC_Runs} valid calculations")
186 if suc_int:
187     mean_dv = np.mean(mc_dvs)
188     std_dv = np.std(mc_dvs)
189     p99_dv = np.percentile(mc_dvs, 99)
190     results = [("Baseline delta V", best["dv_mag"]), ("Mean delta V", mean_dv),
191               ("3 sigma max error", mean_dv + 3*std_dv), ("99th percentile", p99_dv),]
192     print("\nDelta-V Summary:")
193     for label, value in results:
194         print(f"{label}: {value:.3f} km/s")
195
196 plot_monte_carlo(mc_dvs, best["dv_mag"])
197 print()

```

Figure 8: Main simulation script: Monte Carlo campaign execution.

B.2 rusty_lambert.rs

```
1 // Dario Izzo Lamb solver
2 use nalgebra::Vector3;
3 pub fn lambert_solver(r1: Vector3<f64>, r2: Vector3<f64>, tof: f64, mu: f64)
4 -> (Vector3<f64>, Vector3<f64>) {
5
6     ... let r1_mag=r1.norm();
7     ... let r2_mag=r2.norm();
8     ... let c=r2-r1;
9     ... let c_mag=c.norm();
10    ... let s=(r1_mag+r2_mag+c_mag)/2.0;
11    ... let i=r1.cross(&r2).normalize();
12    ... let lambda=i.z.signum()*(1.0-c_mag/s).sqrt();
13    ... let t=tof*(2.0*mu/s.powi(3)).sqrt();
14
15    ... let mut x:f64=0.;
16    ... let tol=1e-11;
17    ... let mut its=6900; //could use the smarter solver
18    ... while its!=0 {
19        ... its-=1;
20        ... let t_calc=eval_tof(x,lambda);
21        ... let err=t_calc-t;
22        ... if err.abs()<tol {break;} // found the fkr
23        ... let dtdx=eval_dt(x,lambda);
24        ... let delta_x=err/dtdx;
25        ... x=x-delta_x;
26    ... }
27
28    ... let y=(1.0-lambda.powi(2)*(1.0-x.powi(2))).max(0.0).sqrt();
29    ... let gamma=(mu*s/2.0).sqrt();
30    ... let rho=(r1_mag-r2_mag)/c_mag;
31    ... let sigma=(1.0-rho.powi(2)).max(0.0).sqrt();
32
33    ... let vr1=gamma*((lambda*y-x)-rho*(lambda*y+x))/r1_mag;
34    ... let vt1=gamma*sigma*(y+lambda*x)/r1_mag;
35
36    ... let vr2=-gamma*((lambda*y-x)+rho*(lambda*y+x))/r2_mag;
37    ... let vt2=gamma*sigma*(y+lambda*x)/r2_mag;
38
39    ... //frame unit vec:
40    ... let ir1=r1/r1_mag; let ir2=r2/r2_mag;
41    ... //let it1=ir1.cross(&ir2).cross(&ir1).normalize();
42    ... //let it2=ir2.cross(&ir1).cross(&ir2).normalize();
43    ... let it1=i.cross(&ir1);
44    ... let it2=i.cross(&ir2);
45    ... let v1=vr1*ir1+vt1*it1; let v2=vr2*ir2+vt2*it2;
```

Figure 9: Izzo Lambert solver in Rust: Main function body.

```

46
47     ... // let mut vi=Vector3::new(0.,0.,0.);
48     ... // let mut vf=Vector3::new(0.,0.,0.);
49     ... // for i in 0..3 {
50     ... //     vi[i]=v1[i];
51     ... //     vf[i]=v2[i];
52     ... // }
53
54     ... (v1, v2)
55 }
56
57 //helper functions
58 fn eval_tof(x: f64, lambda: f64) -> f64 {
59     ... let z=1.0-x*x;
60     ... if z.abs()<1e-5{
61     ...     let d=1e-4;
62     ...     let t_minus=eval_tof_exact(1.0-d,lambda);
63     ...     let t_plus=eval_tof_exact(1.0+d,lambda);
64     ...     let slope=(t_plus-t_minus)/(2.0*d);
65     ...     let t_1=2.0/3.0*(1.0-lambda.powi(3));
66     ...     return t_1+slope*(x-1.0);
67     ... }
68     ... eval_tof_exact(x,lambda)
69 }
70 fn eval_tof_exact(x: f64, lambda: f64) -> f64 {
71     ... let z=1.0-x*x;
72     ... let y=(1.0-lambda*lambda*z).max(0.0).sqrt();
73     ... if x < 1.0 {
74     ...     let arg=(x*y+lambda*z).clamp(-1.0,1.0);
75     ...     (arg.acos()/z.sqrt()-x+lambda*y)/z
76     ... } else {
77     ...     let arg=(x*y-lambda*(-z)).max(1.0);
78     ...     (x-lambda*y-arg.acosh()/(-z).sqrt())/(-z)
79     ... }
80 }
81 fn eval_dt(x: f64, lambda: f64) -> f64 {
82     ... let z=1.-x*x;
83     ... if z.abs()<1e-4 {
84     ...     let h=1e-5;
85     ...     return (eval_tof(x+h,lambda)-eval_tof(x-h,lambda))/(2.0*h);
86     ... let y=(1.0-lambda*lambda*z).max(0.0).sqrt();
87     ... let t=eval_tof_exact(x,lambda);
88     ... (3.*x*t-2.+2.*lambda.powi(3)*x/y)/z
89 }
90

```

Figure 10: Izzo Lambert solver in Rust: Helper functions for time of flight and derivatives.

B.3 plots.py

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 from scipy.integrate import solve_ivp
4
5 def twoB_eci_eom(t, state): #asteroid
6     EMU=398600.4418 #km^3/s^2
7     r = state[0:3]
8     v = state[3:6]
9     r_mag=np.linalg.norm(r)
10    a=-EMU*r/r_mag**3
11    return [v[0],v[1],v[2],a[0],a[1],a[2]]
12 def plot_porkchop(val_ints):
13     if not val_ints: return
14     t_deps=[d['t_dep'] for d in val_ints]
15     tofs=[d['tof_days'] for d in val_ints]
16     dvs=[d['dv_mag'] for d in val_ints]
17     plt.figure(figsize=(10,7))
18     # generate contour
19     contour=plt.tricontourf(t_deps,tofs,dvs,levels=100,cmap='viridis',vmax=3.0)
20     cbar=plt.colorbar(contour)
21     cbar.set_label('delta V (km/s)')
22     best=min(val_ints,key=lambda x:x['dv_mag'])
23     # mark absolute minimum
24     plt.plot(best['t_dep'],best['tof_days'],'r.',markersize=15,label=f"opt: {best['dv_mag']:.3f} km/s")
25     plt.title('earth-asteroid intercept porkchop plot')
26     plt.xlabel('departure time (TU)')
27     plt.ylabel('time of flight (days)')
28     plt.legend()
29     plt.grid(True,alpha=0.3)
30     plt.show()
31
32 def plot_3d_geometry(val_ints, ephem, ast_ist):
33     if not val_ints: return
34     best = val_ints[0]
35     EM_R=384399 #km
36     EM_T=375200 #s
37     dep_time_sec = best['t_dep']*EM_T
38
39     start_pos = next((st[0:3] for t, st in ephem if np.isclose(t, best['t_dep'])), None)
40     if start_pos is None: return
41     initial_sc_state = np.concatenate((start_pos, best['v1_vec']))
42     tof_seconds = best['tof_days']*24*3600
43     total_ast_time = dep_time_sec + tof_seconds
44     sc_arc = solve_ivp(twoB_eci_eom, [0, tof_seconds], initial_sc_state, max_step=3600)
45     ast_arc = solve_ivp(twoB_eci_eom, [0, total_ast_time], ast_ist, max_step=3600)
46
47     lya_pos=np.array([st[0:3] for _, st in ephem])
48     moon_th=np.linspace(0,2*np.pi,300)
49     moon_x=EM_R*np.cos(moon_th); moon_y=EM_R*np.sin(moon_th)
50     t_dep_tu=best['t_dep']
51     moon_pos=np.array([EM_R*np.cos(t_dep_tu), EM_R*np.sin(t_dep_tu), 0.])
52
```

Figure 11: Plotting functions: Porkchop and 3D geometry initialization.

```

53 fig = plt.figure(figsize=(10,10))
54 ax = fig.add_subplot(111, projection='3d')
55 ax.scatter(0, 0, 0, color='blue', s=100, label='Earth')
56 ax.plot(moon_x, moon_y, np.zeros_like(moon_x), color='gray', linewidth=0.5, alpha=0.4)
57 ax.scatter(*moon_pos, color='gray', s=80, label='Moon')
58 ax.plot(lya_pos[:,0], lya_pos[:,1], lya_pos[:,2], color='cyan', linewidth=0.6, alpha=0.5, label='Lyapunov')
59 ax.plot(ast_arc.y[0], ast_arc.y[1], ast_arc.y[2], color='red', linestyle=':', label='Asteroid Trajectory')
60 ax.scatter(ast_arc.y[0,-1], ast_arc.y[1,-1], ast_arc.y[2,-1], color='red', marker='X', s=100, label='Impact')
61 ax.plot(sc_arc.y[0], sc_arc.y[1], sc_arc.y[2], color='green', linewidth=2, label='Spacecraft Trajectory')
62 ax.scatter(start_pos[0], start_pos[1], start_pos[2], color='green', s=50, label='L1 Departure')
63 lim = 1e6
64 ax.set_xlim([-lim, lim])
65 ax.set_ylim([-lim, lim])
66 ax.set_zlim([-lim, lim])
67 ax.set_title('Orbital Intercept Geometry')
68 ax.set_xlabel('X (km)')
69 ax.set_ylabel('Y (km)')
70 ax.set_zlabel('Z (km)')
71 ax.legend()
72 plt.show()
73
74 def plot_monte_carlo(mc_dvs, baseline_dv):
75     if not mc_dvs:
76         print("No Monte Carlo data to plot.")
77         return
78     plt.figure(figsize=(9, 6))
79     plt.hist(mc_dvs, bins=40, color='b', edgecolor='black', alpha=0.7)
80     plt.axvline(baseline_dv, color='red', linestyle='dashed', linewidth=2,
81                label=f'Nominal delta V ({baseline_dv:.3f} km/s)')
82     p99 = np.percentile(mc_dvs, 99)
83     plt.axvline(p99, color='orange', linestyle='dotted', linewidth=2,
84                label=f'99th Percentile ({p99:.3f} km/s)')
85     plt.title('MCS: delta V spread due to tracking errors')
86     plt.xlabel('Required delta V (km/s)')
87     plt.ylabel('Frequency (sims)')
88     plt.legend()
89     plt.grid(True, alpha=0.3)
90     plt.show()

```

Figure 12: Plotting functions: Geometry rendering and Monte Carlo histograms.